Combining IQ Scores

Analysis and a corrected statistical treatment

5 November 2026

The covariance identities reproduced in Ronald K. Hoeflin's article are correct. Their application to qualifying IQ scores requires additional assumptions about the reference population, the composite's distribution, and the meaning of the score. This report distinguishes those assumptions from the algebra, corrects the numerical table, and supplies a corrected replacement article. All displayed numerical examples have been recalculated. A technical appendix illustrates why a standardized composite and an estimate of latent ability need not coincide.

AI disclosure. This report was substantially generated and revised with OpenAI Codex and collaborating AI agents, including its analysis, replacement text, code and verification workflow. Provenance details, human involvement and limitations are disclosed in section D.

1 Analysis of the original article

Hoeflin reproduces George A. Ferguson's treatment of variances and covariances and applies it to combinations of IQ scores. The basic identity

$$\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X,Y) \tag{1}$$

is correct. It determines the variance of a sum without requiring normality. The principal weaknesses arise when a standardized sum is assigned an extreme population percentile and used as a qualifying score.

- **Population calibration.** Correlations among the article's 27 selected high scorers do not establish the correlations in the population used to standardize the tests. Selection can change the relationships, and the appropriate correction depends on how selection occurred (Sackett and Yang, 2000).
- **Distributional assumptions.** Mean zero and variance one do not establish a normal distribution. A one-in-a-million upper-tail claim requires additional justification for the tail model.
- **Numerical correction.** The 100-test entry should be about 3.69, rather than 3.71, standard deviations. The more accurate normal quantile also changes some final digits elsewhere; for example, the four-test result rounds to 3.98 rather than 3.97.
- **Score interpretation.** A standardized composite, a latent ability estimate, and a validated IQ score are distinct quantities. Their equivalence requires evidence about the battery and its intended use (American Educational Research Association et al., 2014, Standard 1.14).
- **Threshold precision.** Rounded whole-number scores are useful illustrations, but are not exact qualifying cutoffs. The scale's standard deviation and the classification rule must be specified.

The replacement below preserves the valid calculation while making its assumptions explicit. The article is newly written; it is not a transcription or a reproduction of the original scans.

2 Corrected article: a statistical method and its limits

Several cognitive test scores can be combined into a single score. The calculation must account for the tests' correlations: correlated scores cannot be treated as independent when calculating the composite's variance. The resulting composite also needs an explicit interpretation. Rescaling it to an IQ-style scale supplies units; evidence about the battery establishes what those units mean.

Before calculating a composite, specify the tests, their weights, the administration procedure, and the population against which scores will be compared. Composite construction and interpretation require their own justification; evidence supporting the individual tests does not automatically establish the meaning of their combination (American Educational Research Association et al., 2014, Standard 1.14).

2.1 Put the component scores on comparable scales

For test i, convert the observed score x_i to a standardized score:

$$z_i = \frac{x_i - \mu_i}{\sigma_i}, \tag{2}$$

where μ_i and σ_i are its mean and standard deviation in the intended reference population. The norms must support comparison with that population. Using different numbers labelled "IQ" does not establish their comparability.

For example, a score of 160 on a scale with mean 100 and standard deviation 15 corresponds to $z = 4$. The same numerical score on a scale with standard deviation 16 corresponds to $z = 3.75$. These scores should be standardized before they are combined.

2.2 Standardize the weighted composite

Let R be the population correlation matrix of the standardized test scores, and let w contain fixed weights. The weighted sum has variance

$$\mathrm{Var}(w^\mathsf{T} Z) = w^\mathsf{T} R w. \tag{3}$$

Consequently, provided this variance is positive, a standardized composite is

$$\boxed{Z_C = \frac{w^\mathsf{T} z}{\sqrt{w^\mathsf{T} R w}}.} \tag{4}$$

With the correct population parameters, this composite has mean zero and variance one. This statement requires finite variances, but does not require normality. When the parameters are estimated, their uncertainty affects the calibration of the composite.

For an equally weighted battery of $n \geq 2$ tests, write $\bar{z}$ for the average standardized score and $\bar{r}$ for the average correlation across the distinct pairs of tests:

$$\bar{r} = \frac{2}{n(n-1)} \sum_{i<j} r_{ij}. \tag{5}$$

Then

$$\begin{aligned}\mathrm{Var}(\bar{Z}) &= \frac{n + 2\sum_{i<j} r_{ij}}{n^2} \\ &= \frac{1 + (n-1)\bar{r}}{n} = \bar{r} + \frac{1-\bar{r}}{n}.\end{aligned} \tag{6}$$

Therefore,

$$\boxed{Z_C = \frac{\bar{z}}{\sqrt{\bar{r} + (1-\bar{r})/n}}.} \tag{7}$$

For a single test, $Z_C = z_1$; no inter-test correlation is needed.

Identical pairwise correlations are unnecessary for equation (6); their average is sufficient for this particular equal-weight composite. The full matrix remains relevant to other weighting schemes and to understanding the battery.

An IQ-style reporting scale with mean 100 and standard deviation s can be defined by

$$I_C = 100 + sZ_C. \tag{8}$$

When all component scores already use mean 100 and the same standard deviation s, the formula becomes

$$I_C = 100 + \frac{\bar{I} - 100}{\sqrt{\bar{r} + (1-\bar{r})/n}}, \tag{9}$$

where $\bar{I}$ is their arithmetic average.

For example, three scores averaging 160, each on an SD-16 scale, produce $\bar{z} = 3.75$. If their population average correlation is 0.60, then

$$Z_C = \frac{3.75}{\sqrt{0.60 + 0.40/3}} \approx 4.3791, \qquad I_C \approx 170.06. \tag{10}$$

This is a score on the specified composite. Interpreting it as a measure of general intelligence requires evidence about what the composite measures.

2.3 Separate standardization from percentile estimation

To assign a percentile, we also need the composite's distribution. Mean zero and variance one do not make a distribution normal. Joint multivariate normality of the component scores is a sufficient condition for a fixed linear composite to be normal. Normal distributions for the individual tests alone are insufficient.

If a standard-normal distribution is justified for the composite, its percentile is $100\Phi(Z_C)$, where Φ is the standard-normal cumulative distribution function. A one-in-a-million upper-tail probability corresponds to

$$q = \Phi^{-1}(0.999999) \approx 4.7534243088. \tag{11}$$

Thus the model-based 99.9999th-percentile threshold is

$$\boxed{\bar{z}_{\min} = q\sqrt{\bar{r} + \frac{1-\bar{r}}{n}}.} \tag{12}$$

The quantile can be reproduced in R with `qnorm(1e-6, lower.tail = FALSE)`; the official documentation defines the normal quantile function and its upper-tail option (R Core Team).

For $\bar{r} = 0.60$, the thresholds are shown in table 1.

Table 1: Model-based 99.9999th-percentile thresholds, assuming a normal composite and average pairwise correlation 0.60.

Number of tests	**Required average component z**	**Required average component score, mean 100 and SD 16**
1	4.7534	176.05
2	4.2516	168.03
3	4.0706	165.13
4	3.9770	163.63
5	3.9198	162.72
10	3.8027	160.84
100	3.6942	159.11
1,000	3.6832	158.93
Limit as n increases	3.6820	158.91

These are thresholds for the *average component score*, under the stated model. The target composite score remains approximately 176.05 on the SD-16 scale in every row. Values are rounded for display; an operational classification rule must specify rounding and use the appropriate attainable scores.

For an SD-15 reporting scale, use $100 + 15\bar{z}_{\min}$ instead. In particular, the single-test threshold would be approximately 171.30. An IQ number without its scale convention is ambiguous.

2.4 Interpret the limiting threshold and correlation sensitivity

The limiting average threshold follows from

$$\lim_{n\to\infty} \bar{z}_{\min} = q\sqrt{\bar{r}}, \tag{13}$$

if the average correlation remains fixed as tests are added. With $\bar{r} = 0.60$, the limiting average component score on the SD-16 scale is approximately 158.91. This is a feature of the assumed sequence of batteries and its threshold; it is not a ceiling on possible composite scores. Adding tests can also change the average correlation, so the fixed-correlation limit should not be treated as a universal rule.

The choice of correlation matters substantially. For three tests, an average correlation of 0.90 raises the required average SD-16 component score to approximately 173.48, compared with 165.13 at 0.60. A smaller assumed correlation gives a larger composite score for the same positive average performance. The correlation therefore needs to be estimated for the relevant population and administration procedure. A lower correlation does not, by itself, establish better measurement; it can also reflect different constructs or greater measurement noise.

2.5 Calibrate the population and the scoring procedure

Correlations in a selected high-scoring sample need not equal correlations in the population used to standardize the tests. Selection can change score variances and relationships. The appropriate treatment depends on the selection mechanism and the information available about the unrestricted population; there

is no universal adjustment. Sackett and Yang (2000) explain why range-restriction corrections depend on those details. A small selected sample cannot directly establish a one-in-a-million population frequency.

The scoring procedure also matters. A fixed battery, the best few results from many attempts, and a rule that keeps adding tests until someone qualifies have different score distributions. Each procedure requires calibration for the rule actually used. Repeated administrations can introduce practice effects, as documented by Hausknecht et al. (2007). Repeated or overlapping tests should not be counted as fresh independent observations without accounting for their dependence and administration history.

Finally, variability across people is different from uncertainty about one person. The correlation matrix determines how the composite varies in the reference population. It does not, by itself, provide an individual confidence interval, separate general ability from shared methods, or establish measurement precision at extreme scores. Those questions require an appropriate measurement model and supporting data.

A defensible report should identify the battery, norms, weights, estimated correlations, and uncertainty. Direct composite norms should be used when they adequately support the intended interpretation. If an extreme percentile comes from a distributional model beyond the available normative evidence, report it as a model-based extrapolation and show how sensitive it is to the assumptions. The testing standards require clear descriptions of reference populations and norming procedures in Standards 5.8–5.9, and justification for score interpretations in Standards 1.1–1.4 (American Educational Research Association et al., 2014).

A Composite scores and latent ability: a mathematical illustration

To see why these quantities differ, suppose, expressly as a hypothetical model, that

$$Z_i = \sqrt{\rho}\, G + \sqrt{1-\rho}\, \varepsilon_i, \qquad 0 < \rho < 1, \tag{14}$$

where G and all ε_i are independent standard-normal variables. This model treats all shared variance as one common factor and all residuals as mutually independent. Real batteries need not satisfy those assumptions.

In this particular model, the equal-weight composite's squared correlation with G is

$$\mathcal{R} = \mathrm{Corr}(G, Z_C)^2 = \frac{n\rho}{1 + (n-1)\rho}. \tag{15}$$

The conditional distribution of the latent factor has

$$\mathrm{E}[G \mid Z_1, \ldots, Z_n] = \sqrt{\mathcal{R}}\, Z_C, \tag{16}$$

$$\mathrm{Var}(G \mid Z_1, \ldots, Z_n) = \frac{1-\rho}{1 + (n-1)\rho}. \tag{17}$$

These identities follow from Gaussian conditioning. In particular, $\mathrm{Cov}(G, \bar{Z}) = \sqrt{\rho}$ and $\mathrm{Var}(\bar{Z}) = \rho + (1-\rho)/n$; the residual contrasts contain no additional information about G under this model.

For three tests with $\rho = 0.60$, a composite at $Z_C = 4.7534$ has conditional mean G of approximately 4.30 and conditional standard deviation approximately 0.43. On a hypothetical latent scale with mean 100 and SD 16, these become a mean of approximately 168.79 and a standard deviation of 6.82. The composite's reporting-scale value is approximately 176.05. The two values answer different questions, even in this unusually simple model.

The illustration is a mathematical counterexample to treating a standardized composite as automatically identical to a latent-ability estimate. It is not a proposed correction factor for actual IQ scores, and inter-test correlation should not generally be equated with reliability or common-factor variance.

B Numerical reproduction

The accompanying script `scripts/verify_calculations.py` recalculates the normal quantile, checks the table against `data/thresholds.csv` and the numerical entries in this source, and verifies the worked examples. It uses the upper-tail probability directly through `scipy.stats.norm.isf(1e-6)`. The mathematical results do not depend on the choice of software.

C Expanded software verification and its limits

The original verification selected only the arithmetic core. That boundary was broader than necessary: the calculation and simulation drivers and most numerical orchestration can be analyzed in SPARK. The expanded implementation makes `Hoeflin_Calculations`, `Hoeflin_Simulation`, and the algorithm bodies in `Normal_Support` SPARK-on, alongside the original arithmetic in `Hoeflin_Core`. Argument parsing and CSV rendering are isolated in `Report_IO`, whose SPARK interface explicitly models external state. The proof project enumerates all ten source files, including the trusted I/O completion, rather than selecting only the core.

The Five-contract computer10 evidence report records source hashes, commands, tool identities, native reports, coverage and numerical comparisons from 4 November 2026. The symbolic-check follow-up adds fresh Python verification and re-audits these unchanged native records; it does not claim new prover invocations. Earlier 47-check and 110-test results remain immutable historical snapshots, not counts for the expanded implementation.

C.1 What is now analyzed

SPARK now covers the inverse-survival bisection loop, exact 64-bit random state recurrence, open-unit conversion, Box–Muller composition, common-factor simulation, signed and squared accumulations, event-count updates, and both main procedures. Random generation uses procedures with explicit state updates instead of side-effecting functions. Proved loop invariants and small SPARK helpers bound accumulation; the exact integer-to-floating successor lemma is proved, not imported as an axiom. The original arithmetic core still establishes exact integer variance bookkeeping, bounded floating-point transforms, and nested event counts with specified increments.

The 4 November five-prover portfolio discharged **404 native checks: 129 flow checks and 275 prover checks**, with zero unproved and zero justified checks. CVC5, Z3, Alt-Ergo, classic COLIBRI, and Yices were requested together. This result does not imply that every solver independently completed every obligation. The deliberately incorrect inclusion-exclusion control was rejected with an unproved functional contract. Fresh source copies and object directories prevent reuse of stale proof results. The compiler identifies itself as GNAT 13.2.0; the matching GNATprove banner is `0.0w`. Exact executable identities are retained in the manifest.

Native coverage records 53 `all`, 14 `spec`, and zero `no` entities. Here `all` describes a SPARK specification and body, or a declaration requiring no body; it is not itself a proof result. The `spec` entries expose trusted completions. In the earlier source, whole off-mode units emitted empty entity arrays, so zero `no` entries must never be interpreted as zero excluded code. The retained flow-only baseline has nine `all` entities and is explicitly not a new proof campaign. Native proof totals and coverage statuses are audited separately.

The first attempt left the newly added driver receipt postcondition unproved. A fresh full-project retry added outer-loop receipt invariants and discharged every check. The initial failed result and source remain recorded. The final manifest combines actual successful invocations and fresh source-pinned structural audits; it is not a claim that the initial launcher exited successfully.

C.2 Trusted numerical and I/O boundaries

Only five small numerical completions in `Normal_Support` remain off: `Square_Root`, `Survival`, `Check_Inverse_Bracket`, `Rare_Quantile`, and `Checked_Radius`. Their bodies check each promised normal-return range or relationship and raise on rejection; they do not clamp values. The foreign C `erfc` import and `Report_IO`'s parsing, formatting, output, and exceptions remain trusted. The installed Ada elementary-function specifications are SPARK-compatible, but their limited contracts and unproved implementations remain part of the proof boundary.

Consequently, a caller proof is conditional on these interfaces: it does not prove numerical backend accuracy, that a guard cannot reject, or that I/O succeeds. Bisection bounds do not prove survival-function monotonicity, inverse convergence, or an approximation-error bound. No project `pragma Assume`, imported proof axiom, or suppressed check is used. Zero explicit assumptions or suppressions in native metadata does not eliminate trust in off-mode completions or library contracts.

C.3 Five explicit contracts per subprogram

Every current SPARK function and procedure, including local and Ghost helpers and trusted interfaces, has at least five explicit entries. This project inventories 35 functions and 28 procedures; the full repository has 164 subprograms and 923 entries, including both negative controls. The audit counts top-level `Pre/Post` conjuncts, `Contract_Cases` alternatives and explicit `Global/Depends` aspects. Mode flags, subtype declarations, body assertions, duplicates and literal Boolean padding do not count. The entries can overlap logically; five does not mean five independent guarantees.

The checker handles this project's restricted Ada subset and cross-checks native locations and `all/spec` statuses; GNAT checks syntax and conformance. This structural floor does not certify specification adequacy, trusted implementation refinement or statistical validity.

Python requires at least five distinct real Deal decorators per named function, expressing appropriate domain, result, exception, side-effect or preservation properties. There is no upper limit: regression cases accept five, six, seven and eight distinct decorators and reject zero through four. Repeated normalized syntax cannot pad the count.

C.4 Ghost models for the remaining interfaces

`Hoeflin_Boundary_Models` is an implemented SPARK model package. Its pure predicates, projection types and receipt transitions are ordinary SPARK; observation and lexical-check procedures remain explicitly Ghost. It defines acceptance predicates for the five excluded numerical completions, a proved scaling model for an accepted C `erfc` value, and explicit observations of the actual sine and cosine results in `Normal_Pair`. The trigonometric model records the range $[-1, 1]$ under the installed library specification; it does not establish a trigonometric identity, rounding bound, or backend correctness.

Excluded numerical bodies call Ghost observation procedures after their ordinary checks. An observation procedure's precondition is checked when Ghost is enabled; its analyzed body and model do not prove that a call inside an excluded body always meets that precondition. The original ordinary contracts and guards are retained. Ghost checking can be disabled while ordinary assertions remain enabled, without placing Ghost-dependent expressions inside an ordinary postcondition.

The ordinary output receipt records an event, typed payload, nominal column count, section, and whether the latest request is admissible. Its counters are modular, and it is a bounded receipt summary rather than a complete character history. The receipt update remains active under Ghost Ignore so the main procedures can state executable normal-return postconditions. The admissibility flag is observed, not a theorem that

all external calls obey a protocol. Models also cover canonical decimal integer images, requested numeric value and precision, and label length and separator counts. The numeric projection does not prove correct rounding or actual printed characters. The I/O implementation's refinement of these models, physical output, and exception behavior remain unproved.

The source-pinned register maps every native specification-only entity, the foreign import, and all three private formatting helpers to actual Ghost observations or ordinary receipt updates and analyzed model entities. Standard-library dependencies and unproved obligations are also inventoried. Separate sine and cosine links are checked against their actual cached assignments and observer calls. The register is a coverage and provenance audit, not a proof of model adequacy. The deliberately wrong Ghost event-counter postcondition must be rejected, in addition to the original incorrect union control.

The 4 November builds with Ghost enabled and disabled retain ordinary assertion checking. Both policies reproduce the original million-trial CSVs byte for byte, and the validators and replay pass in normal and optimized Python. Representative decimal, signed, whitespace, underscore, based-literal, and exponent forms of the existing Ada option grammar are checked under both policies. These finite erasure regressions complement SPARK's Ghost rules; they do not prove noninterference or totality of excluded bodies. The role and erasure rules of Ghost code are described in AdaCore's specification guide.

Runtime checks exposed a failed assertion in the installed GNAT 13.2.0 build at `-O2`, for a classification postcondition written with Boolean equivalences. An isolated test of all four Boolean inputs passed at `-O0` and with GNAT 14.2.0. The equivalent guarded conditions pass in the required GNAT 13.2.0 `-O2` build, with the function body unchanged. The failed attempt and compatibility probes are retained. Compiler correctness remains part of the trusted infrastructure; finite compatibility tests are not a compiler proof.

C.5 Numerical regression, Deal, and audit results

The simulation uses 1,000,000 trials per case, resetting seed `20261030` across the eight cases. Replay matches all tail counts exactly; moments meet the validators' tolerances. CLI boundary tests also pass. Numerical equivalence of the core's evaluation orders is checked against the report formulas; bit identity is claimed only for the CSV comparisons.

All 28 current Python files import Deal. All 292 named functions, including tests and evidence tools, carry at least five distinct decorators: 1,460 in total, with zero gaps. All 27 positive files are lint-clean; the negative fixture gives exactly the expected `DEL012`. All 343 tests, numerical scripts, contract coverage and historical-source audits pass normally and under optimization; CLI contracts remain enabled. The version-pinned symbolic gate proves three correct theorems and rejects the unchanged negative fixture with a counterexample. Six postconditions prove and one fails as expected, with no skips. Its integer-type model preserves Boolean subtype semantics; restricted chain expansion retains both bounds. Unsupported forms fail closed. The adapter and normalizer are trusted tooling, not proofs of arbitrary Python, libraries or the full programs. Original contracts and dependencies are unchanged.

The auditor checks required projects and SPARK-on bodies, native totals, coverage, source identities, checksums and negative controls. Historical Ada requires an explicit matching source tree; historical Python may use verified immutable Git blobs. Audits do not rerun provers or independently validate SMT proof objects.

Common random numbers make the eight simulation cases dependent. Replay does not establish independent draws; rare counts at this sample size do not validate a one-in-a-million tail. Neither Deal nor SPARK proves the Gaussian common-factor model, empirical covariance calibration, an identification of correlation with reliability, or a psychometric interpretation. The distinction between a standardized composite and latent ability remains essential.

D AI disclosure and provenance

This report was substantially generated and revised using OpenAI Codex and collaborating AI sub-agents. The session identifies Codex as based on the GPT-6 family. An exact serving checkpoint, per-agent model identifiers, and generation settings were not preserved as versioned provenance; no more precise model attribution is claimed. This disclosure applies to the analysis, corrected replacement text, project-authored software, and earlier revisions, as well as the present additions.

AI contributions included interpreting the supplied articles, drafting and revising the mathematical exposition and statistical discussion, selecting and checking numerical examples, researching references, designing and editing the Python/Deal and Ada/SPARK programs, ghost models, tests and evidence auditors, orchestrating verification, typesetting and inspecting the PDFs, and preparing repository commits. The human requester supplied the source articles, specified the objectives, formats and verification requirements, requested revisions, and authorized repository publication. The recorded audits and parallel reviews were performed by AI agents and automated tools. Independent human peer review or professional sign-off is not recorded and is not claimed. Repository ownership and Git commit attribution do not identify passages as human-authored.

The original authors remain credited as sources. The replacement text is an AI-generated analytical reconstruction prepared for this project; no endorsement by the original authors, their hosts, or cited organizations is claimed. Sources were consulted at different levels, including article text, software documentation and bibliographic metadata. Per-reference notes identify relevant access limits; full-text review of every cited work is not claimed.

Calculations, tests and prover invocations were actually executed on `computer10`; the retained commands, native outputs, source hashes, tool versions and checksums provide the verification evidence. These are machine-checker results under the documented contracts and assumptions, not evidence supplied solely by an AI assertion. Here "independent" validation or replay means a separate implementation, algorithm or library; it does not mean independent human authorship. The implementations, contracts, tests and their reviews were developed within the same AI workflow. Formal proof covers the encoded properties; correctness of the specifications, excluded implementation refinement and statistical-model adequacy remain distinct obligations. Ghost models do not establish correctness of foreign numerical libraries or physical I/O.

No new participant data were collected and no empirical psychometric validation study was conducted for this project. Simulations use synthetic draws under the stated mathematical model. The repository preserves source and evidence history, but not a complete prompt/tool transcript, exact per-line authorship, or an AI-contribution percentage. Those details cannot be reconstructed reliably and have not been invented. The disclosure is current through 5 November 2026.

References

American Educational Research Association, American Psychological Association, and National Council on Measurement in Education. *Standards for Educational and Psychological Testing*. American Educational Research Association, Washington, DC, 2014. ISBN 978-0-935302-35-6. URL https://www.testingstandards.net/uploads/7/6/6/4/76643089/9780935302356.pdf.

John P. Hausknecht, Jane A. Halpert, Nicole T. Di Paolo, and Meghan O. Moriarty Gerrard. Retesting in selection: A meta-analysis of coaching and practice effects for tests of cognitive ability. *Journal of Applied Psychology*, 92(2):373–385, 2007. doi: 10.1037/0021-9010.92.2.373. URL https://ecommons.cornell.edu/entities/publication/97becf46-3c0a-4de4-a48f-79b715ba8446.

Ronald K. Hoeflin. The statistical technique for combining IQ scores. Noesis, issue 141; reprinted from The Circle. URL https://megasociety.org/noesis/141/ferguson.html. Includes an excerpt from George A. Ferguson, Statistical Analysis in Psychology and Education, third edition, pp. 103–105. Accessed 30 October 2026.

R Core Team. *The Normal Distribution*. R Foundation for Statistical Computing. URL https://stat.ethz.ch/R-manual/R-devel/library/stats/html/Normal.html. R stats package reference documentation. Accessed 30 October 2026.

Paul R. Sackett and Hyuck Yang. Correction for range restriction: An expanded typology. *Journal of Applied Psychology*, 85(1):112–118, 2000. doi: 10.1037/0021-9010.85.1.112. URL https://pubmed.ncbi.nlm.nih.gov/10740961/.